Елементи алгоритмічної теорії графів

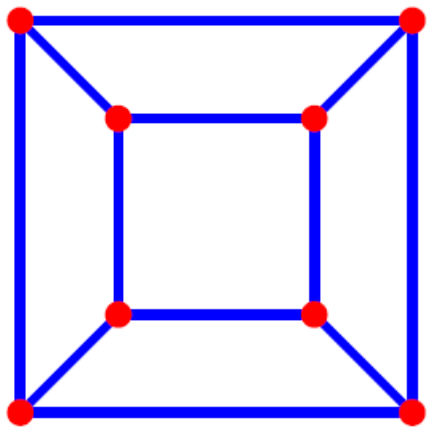
ЛЕКЦІЯ 8

ЩО ТАКЕ ГРАФ?

(Неорієнтованим) графом називається пара (V, E), де

✓V – множина вершин графа;

✓$E \subseteq C_{|V|}^{2} = \{\{u, v\} \subseteq V \mid u \neq v\}$ – множина ребер.



ЩО ТАКЕ ГРАФ?

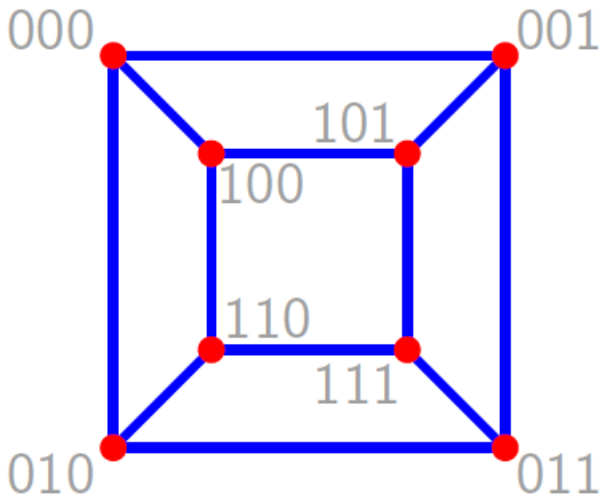
(Неорієнтованим) графом називається пара (V, E), де

✓V – множина вершин графа;

✓$E \subseteq C_{|V|}^{2} = \{\{u, v\} \subseteq V \mid u \neq v\}$ – множина ребер.

$V = \{000, 001, \ldots, 111\}$

$\{u, v\} \in E \quad \Leftrightarrow \quad ?$



ЩО ТАКЕ ГРАФ?

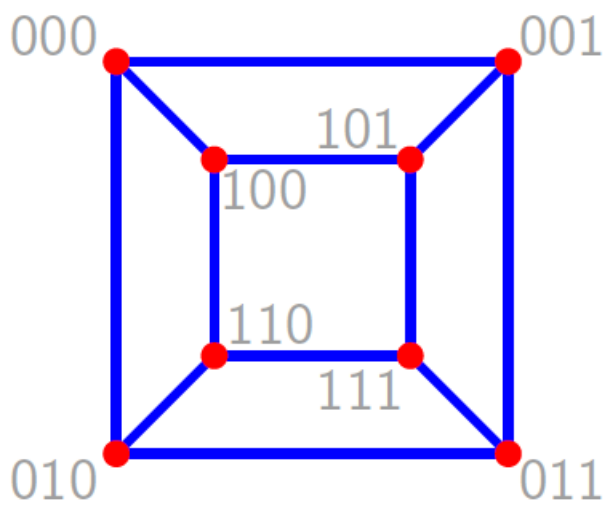
(Неорієнтованим) графом називається пара (V, E), де

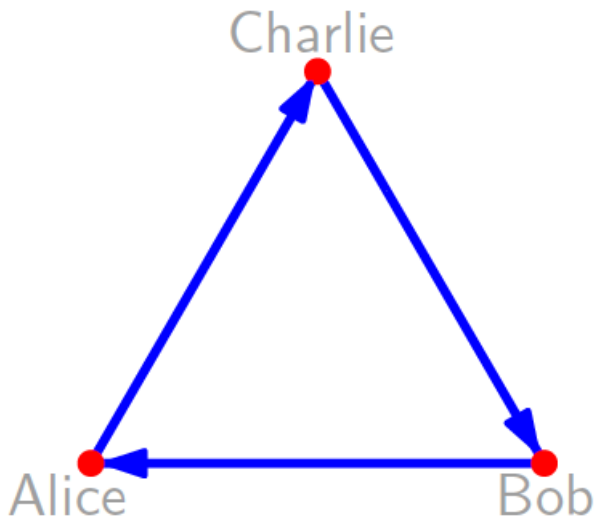
✓V – множина вершин графа;

✓$E \subseteq C_{|V|}^{2} = \{\{u, v\} \subseteq V \mid u \neq v\}$ – множина ребер.

$V = \{000, 001, \ldots, 111\}$

$\{u, v\} \in E \quad \Leftrightarrow \quad ?$





ЩО ТАКЕ ГРАФ?

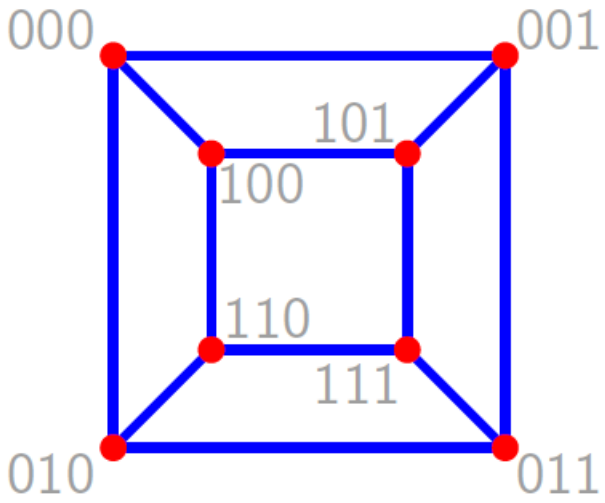
Орієнтованим графом називається пара (V, E), де

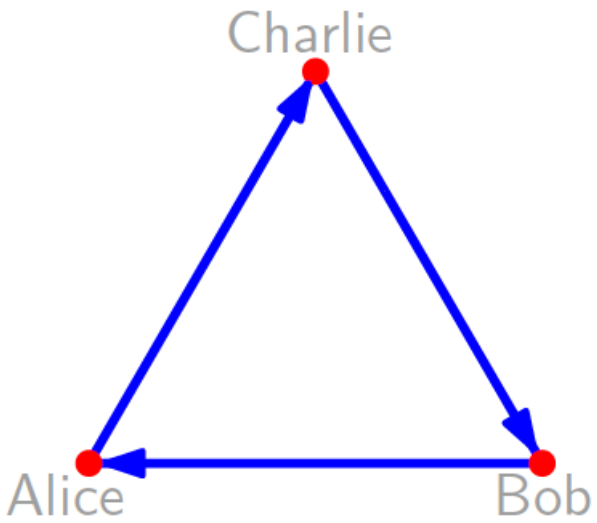
✓V – множина вершин графа;

✓$E \subseteq V \times V = \{(u, v) \in V \times V | u \neq v\}$ – множина ребер.

$V = \{000, 001, \ldots, 111\}$

$\{u, v\} \in E \quad \Leftrightarrow \quad ?$





ВИДИ ГРАФІВ

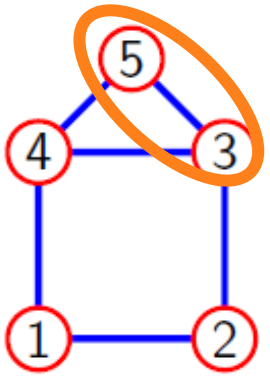
- *Мультиграфом* називають граф, у якому кожна довільна пара вершин може бути з'єднана більше, ніж одним ребром.

Очевидно, що мультиграфи можуть бути як орієнтованими так і неорієнтованими.

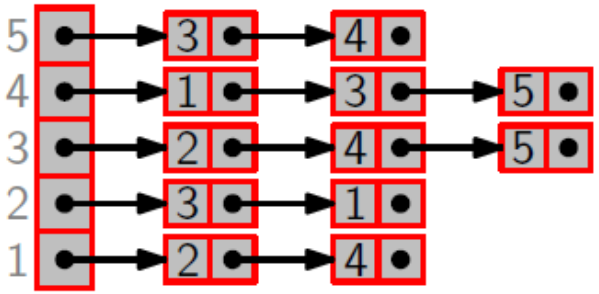
Планарним називається граф, який ізоморфний до графа, який зображається на площині так, що жодні його два ребра геометрично не перетинаються у жодній точці, за винятком інцидентних їм вершин.

СПОСОБИ ПОДАННЯ ГРАФІВ

Будемо говорити, що вершини 3 та 5 є суміжними



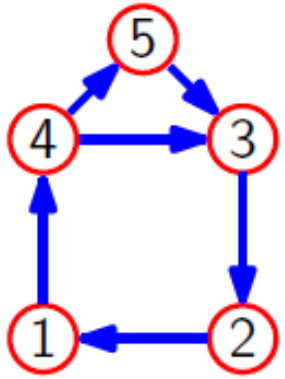
Неорієнтований граф



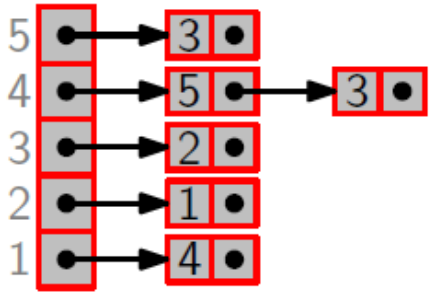
Список суміжності
(adjacency list)
$Adj[i] = \{j \in V | \{i,j\} \in E\}$

	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	0
3	0	1	0	1	1
4	1	0	1	0	1
5	0	0	1	1	0

Матриця суміжності



Орієнтований граф

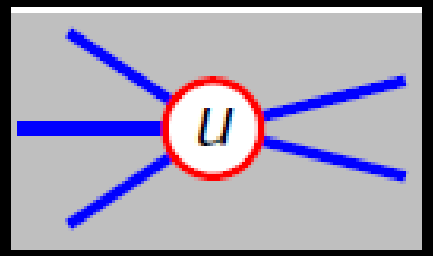


$Adj[i] = \{j \in V | (i,j) \in E\}$

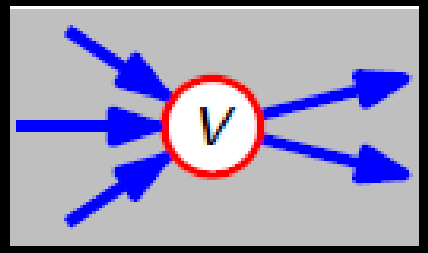
	1	2	3	4	5
1	0	0	0	1	0
2	1	0	0	0	0
3	0	1	0	0	0
4	0	0	1	0	1
5	0	0	1	0	0

$a_{ij} = 1 \Leftrightarrow (i,j) \in E$

СТЕПЕНІ ВЕРШИН



$\deg u = |Adj[u]|$



$\text{outdeg}\, v = |Adj[v]|$

$\text{indeg}\, v = |\{u \in V | (u, v) \in E\}|$

У неорієнтованому графі **півсума** степенів усіх вершин рівна кількості ребер.

У неорієнтованому графі кількість вершин з непарним степенем є парною.

NETWORKX

NetworkX - це модуль Python для моделювання та аналізу графів та мереж.

Якщо ви використовуєте Anaconda, NetworkX вже встановлено. В інших випадках можна легко встановити NetworkX за допомогою диспетчера пакетів (наприклад pip install networkx).

Документація: http://networkx.github.io.

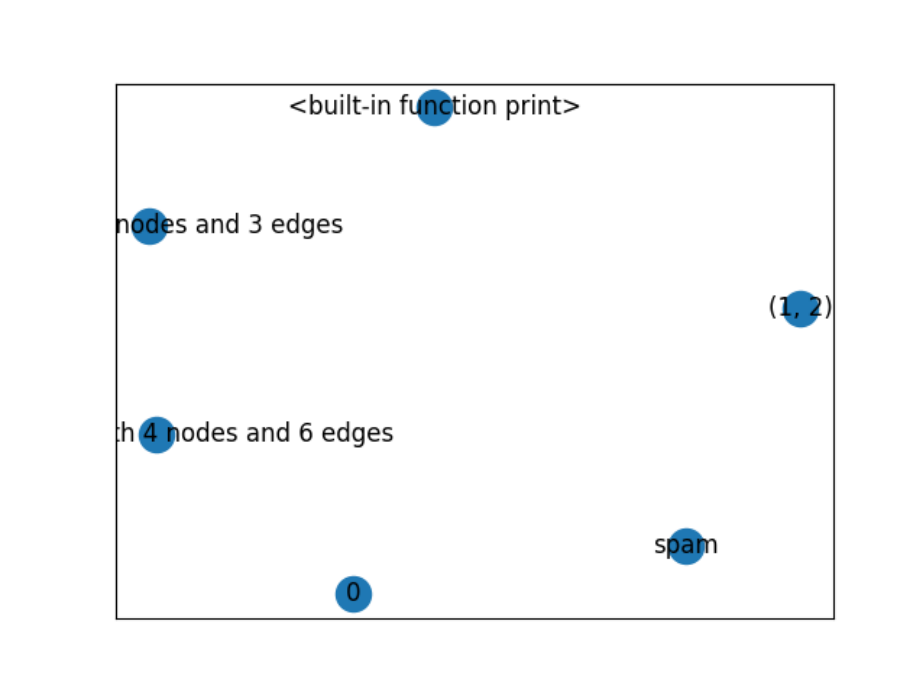
Можливо, варто згадати, чому ми вибираємо NetworkX над іншими варіантами. По-перше, всі його функції написані на Python, а їх вихідні коди доступні на веб-сайті вище, тому можна перевірити деталі їх алгоритмів і навіть змінити коди, якщо це потрібно. По-друге, NetworkX використовує прості «словники» Python для зберігання інформації про графи чи мережі, які є повністю доступними та гнучкими для зберігання додаткової інформації. Ця властивість особливо важлива, при реалізації імітаційних моделей динамічних систем. Спеціалізовані структури даних, реалізовані в інших інструментах мережевого аналізу, можуть не мати такої гнучкості.

СТВОРЕННЯ ГРАФІВ

```
import networkx as nx

# створення порожніх графів

G = nx.Graph()  # Неорієнтований граф
DG = nx.DiGraph()  # Орієнтований граф
MG = nx.MultiGraph()  # Неорієнтований мультиграф
MDG = nx.MultiDiGraph()  # Орієнтований мультиграф
```

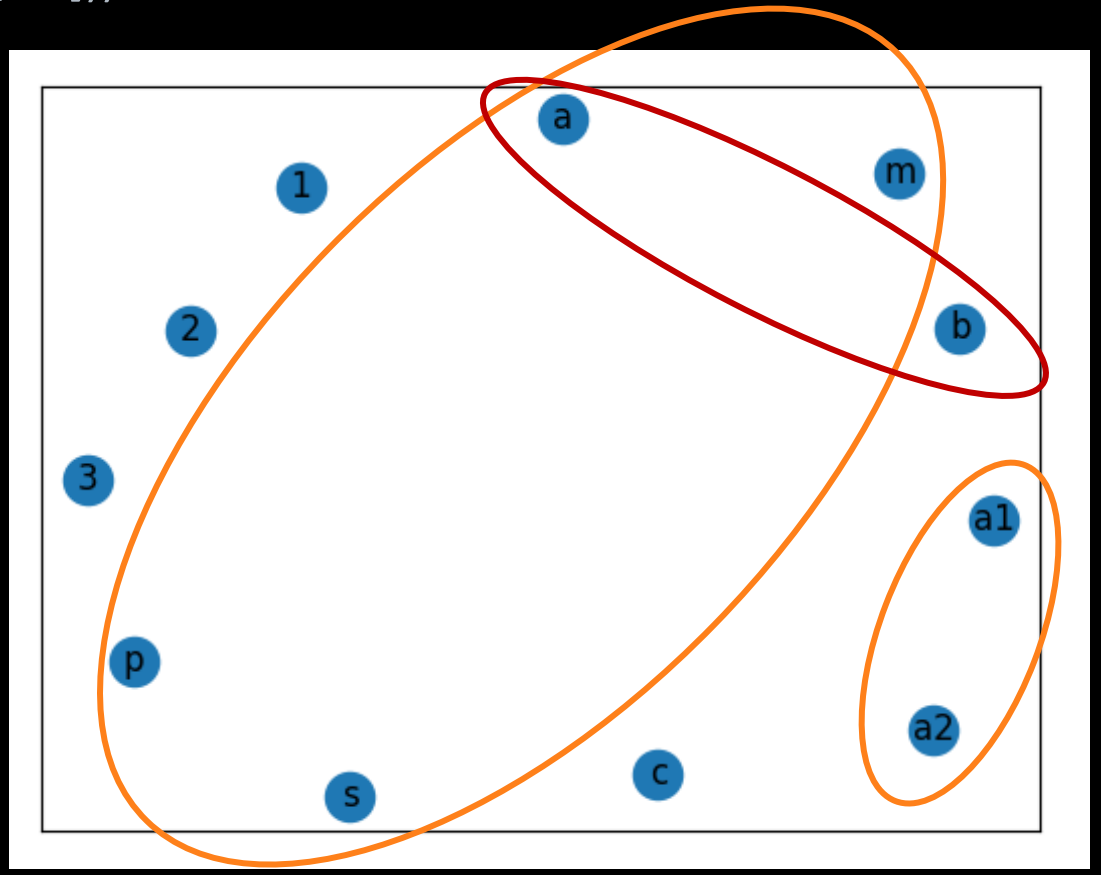
СТВОРЕННЯ ГРАФІВ

```
G = nx.Graph()
K3 = nx.Graph(([1,2], [2,3], [3,1]))  # повний граф
W3 = nx.MultiDiGraph(([1,2], [2,3], [3,1], [1,'c'], [2,'c'], [3, 'c']))  # колесо

# додавання вершини
G.add_node(0)  # будь-який об'єкт, крім None
G.add_node(print)
G.add_node('spam')
G.add_node((1, 2))
G.add_node(K3)
G.add_node(W3)
# G.add_node(None)  # ValueError

nx.draw_networkx(G, with_labels=True)
plt.show()
```



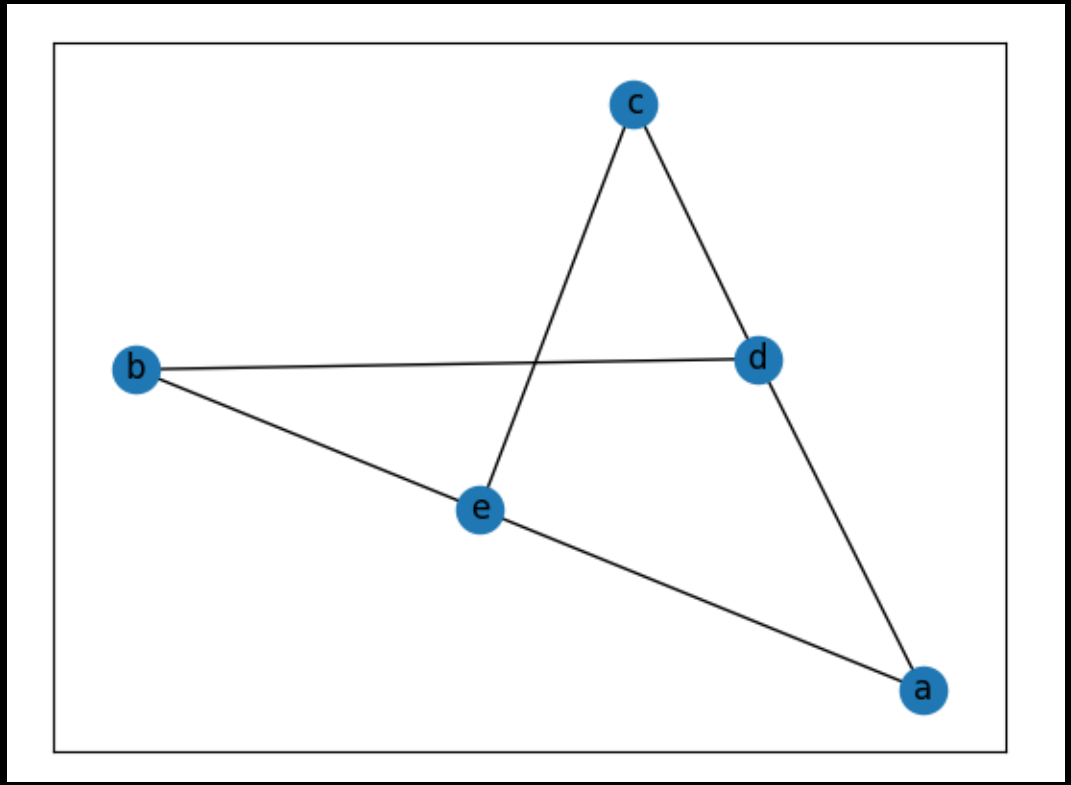
СТВОРЕННЯ ГРАФІВ

```
G = nx.Graph()
K3 = nx.Graph(([1,2], [2,3], [3,1]))  # повний граф
W3 = nx.MultiDiGraph(([1,2], [2,3], [3,1], [1,'c'], [2,'c'], [3, 'c']))  # колесо

# додавання багатьох вершин
G.add_nodes_from(['a1', 'a2'])
G.add_nodes_from('spam') # додає всі літери
G.add_nodes_from({'a': 2, 'b': 4})  # додає ключі
G.add_nodes_from(K3)
G.add_nodes_from(W3)


nx.draw_networkx(G, with_labels=True)
plt.show()
```



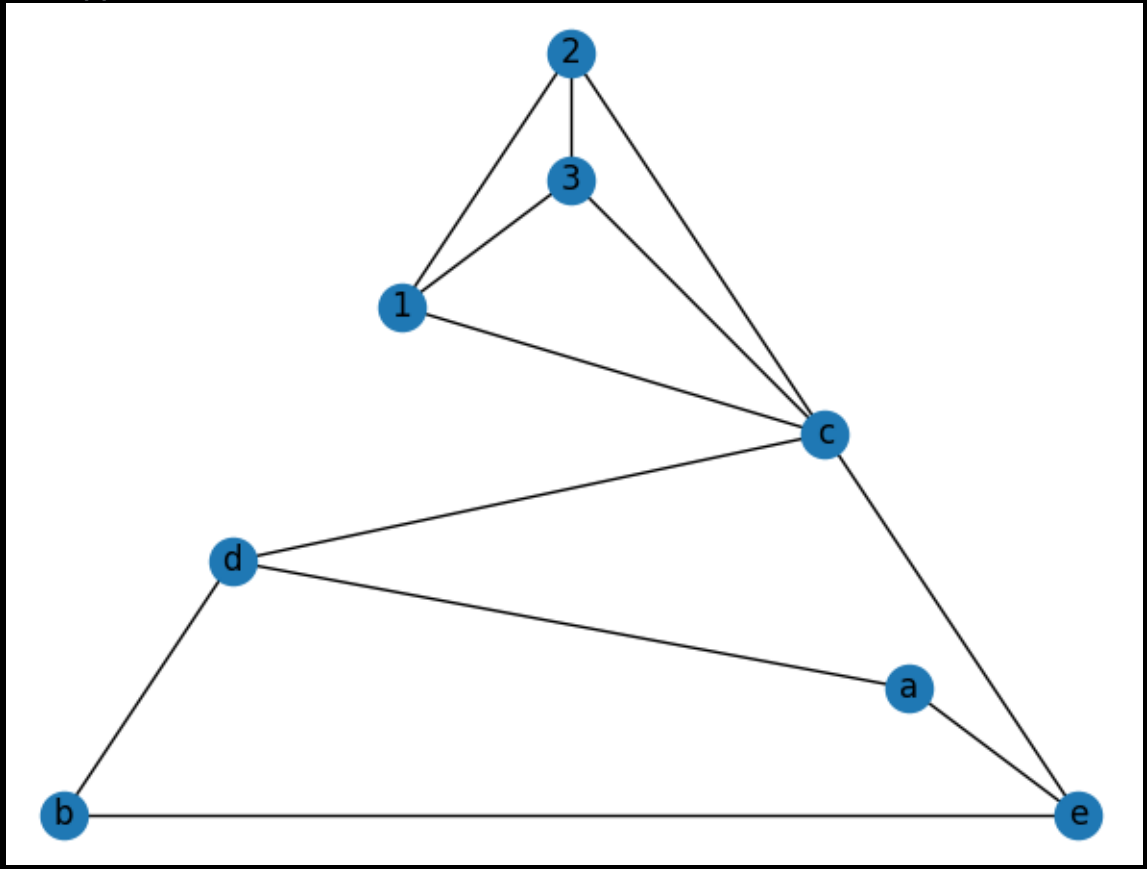
СТВОРЕННЯ ГРАФІВ

```
G = nx.Graph()

# додавання ребер по одному
G.add_nodes_from('abcde')
G.add_edge('a', 'd')

# Додавання групи ребер
G.add_edges_from([(i, j) for i in 'abc' for j in 'de'])

nx.draw_networkx(G, with_labels=True)
plt.show()
```



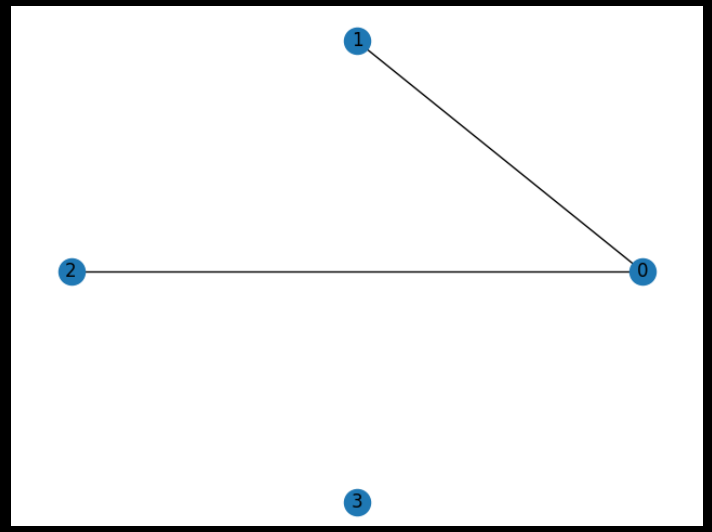
СТВОРЕННЯ ГРАФІВ

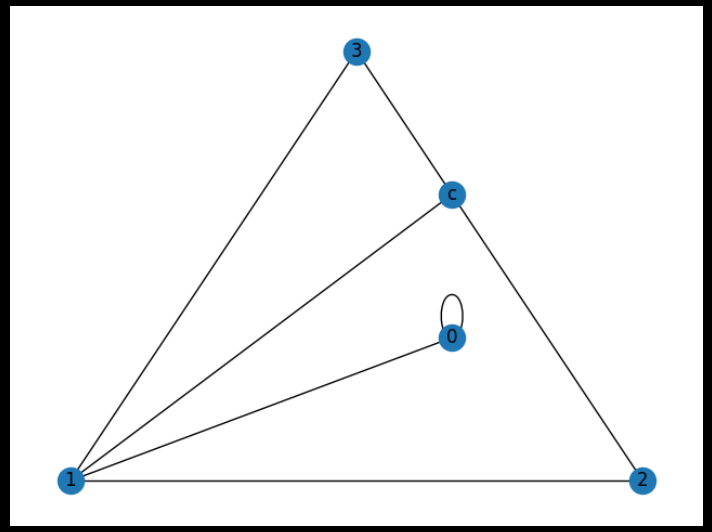
```
W3 = nx.Graph(([1,2], [2,3], [3,1], [1,'c'], [2,'c'], [3, 'c']))  # колесо
G = nx.Graph()

# додавання ребер по одному
G.add_nodes_from('abcde')
G.add_edge('a', 'd')

# Додавання групи ребер
G.add_edges_from([(i, j) for i in 'abc' for j in 'de'])
G.add_edges_from(W3.edges)
```



СТВОРЕННЯ ГРАФІВ

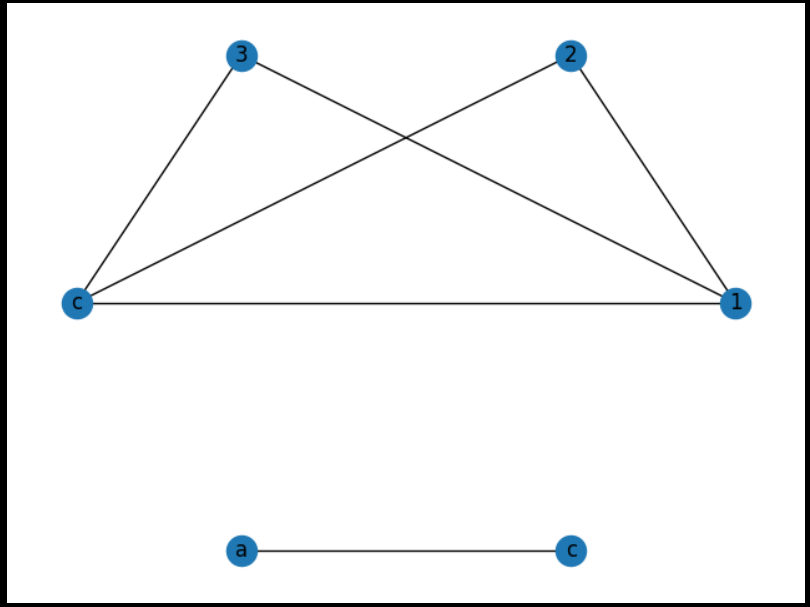
```
W3 = nx.Graph(([1, 2], [2, 3], [3, 1], [1, 'c'], [2, 'c'], [3, 'c']))  # колесо
G = nx.Graph()

W3.remove_edge(2, 3)  # видалення ребра 2-3
W3.add_edges_from([(1, 0), (0, 0)])
print(W3.adj)
# {1: {2: {}, 3: {}, 'c': {}, 0: {}}, 2: {1: {}, 'c': {}}, 3: {1: {}, 'c': {}}, 'c': {1: {}, 2: {}, 3: {}}, 0: {1: {}, 0: {}}}


# створення через список суміжності
adj_list = {0: [1, 2], 1: [0], 2: [0], 3: []}
G1 = nx.from_dict_of_lists(adj_list)
G2 = nx.from_dict_of_dicts(W3.adj)

nx.draw_circular(G1, with_labels=True)
plt.show()

nx.draw_planar(G2, with_labels=True)
plt.show()
```





АТРИБУТИ ГРАФІВ

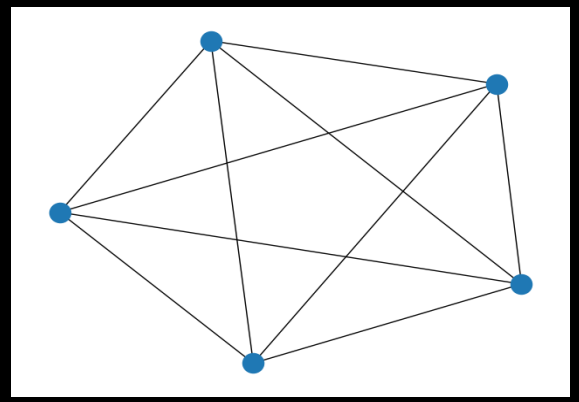
```
G = nx.Graph(([1, 2], [3, 1], [1, 'c'], [2, 'c'], [3, 'c']))
print(G.edges)  # [(1, 2), (1, 3), (1, 'c'), (2, 'c'), (3, 'c')]
print(G.nodes)  # [1, 2, 3, 'c']
print(G.number_of_edges())  # 5
print(G.number_of_nodes())  # 4
print(G.degree)  # [(1, 3), (2, 2), (3, 2), ('c', 3)]
```

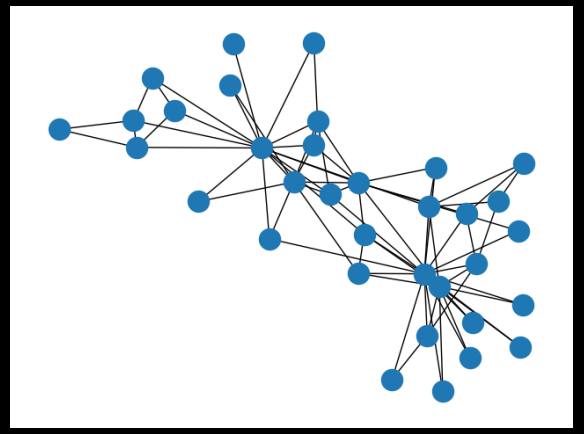


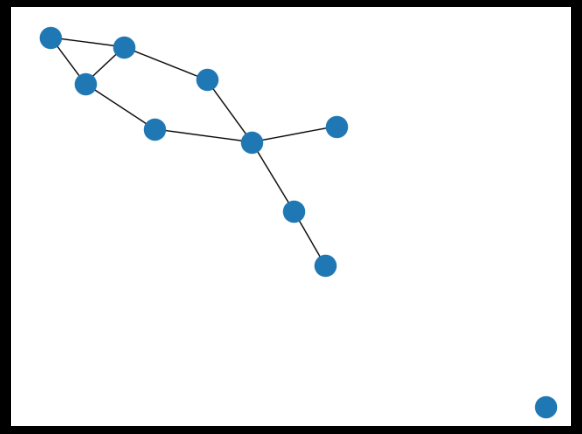

```
# додамо зважені ребра
G.add_weighted_edges_from([(1, 2, 0.4), ('a', 'c', 0.8)])
print(G.edges[1,2]['weight'])  # 0.4 доступ до ваги на ребрі
```

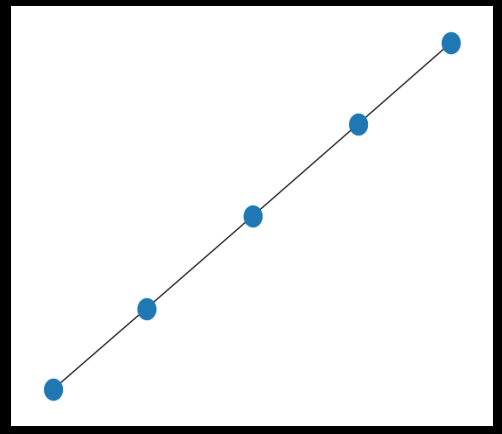
ГЕНЕРАТОРИ ГРАФІВ

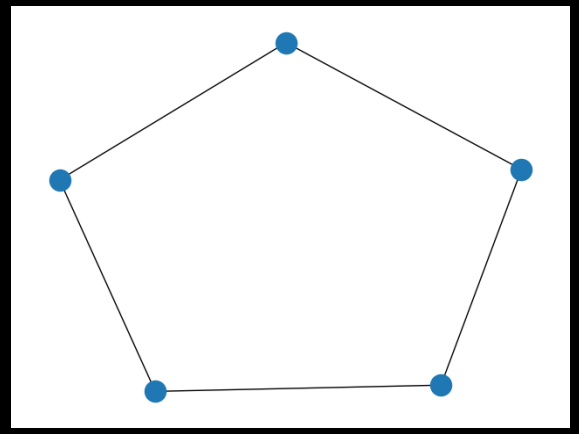
```
K5 = nx.complete_graph(5)  # Повний граф
Ch5 = nx.path_graph(5)      # Ланцюг
C5 = nx.cycle_graph(5)     # Цикл
Rand = nx.erdos_renyi_graph(10, 0.3)  # Випадковий граф
kar = nx.karate_club_graph()  # карате клуб
```

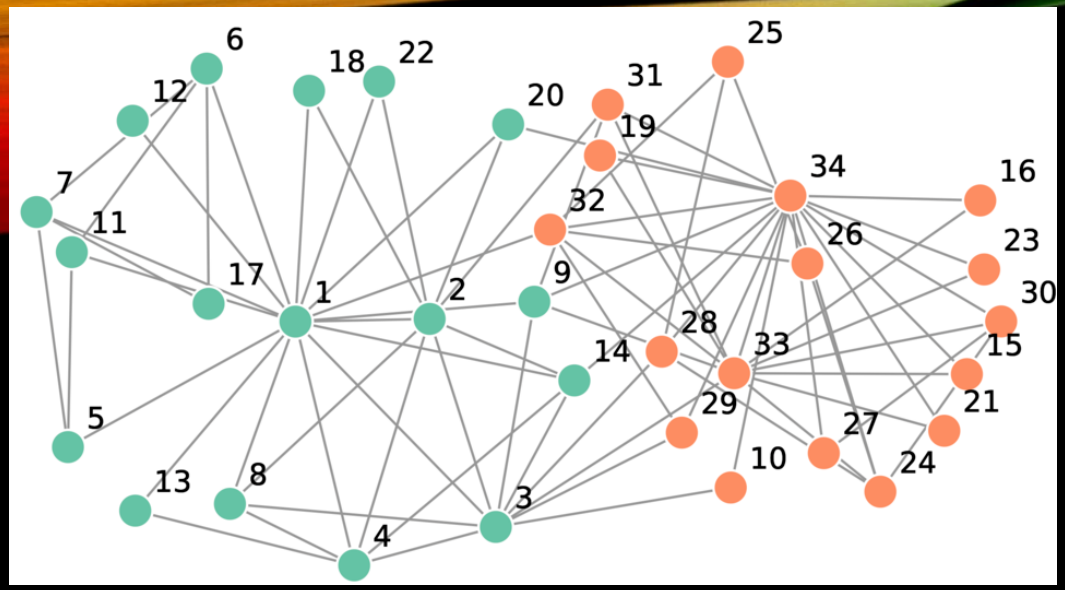












МОДЕЛЬ КЛУБУ КАРАТЕ ЗАХАРІ (ZACHARY'S KARATE CLUB)

Граф **Zachary's Karate Club** — це **реальна соціальна мережа**, яку дослідив соціолог Уейн Захарі у 1977 році. Вона описує **взаємодії між 34 членами клубу карате** в американському університеті.

Суть моделі:

Кожна вершина — учасник клубу.

Кожне ребро — соціальна взаємодія або дружба між двома людьми.

Загалом у графі: 34 вершини78 ребер

У клубі згодом виник конфлікт між тренером (node 1) і адміністратором (node 34), що призвів до розколу клубу на 2 групи.

МОДЕЛЮВАННЯ ДИНАМІКИ МЕРЕЖ ЗА ДОПОМОГОЮ ГРАФІВ

Взагалі існує багато різних класів задач для моделювання динаміки мережевих моделей. Ми обговоримо лише одну з них.

Задача про моделі для «динаміки в мережах»

Ці моделі є найбільш природним продовженням традиційних моделей динамічних систем. Вони розглядають, як змінюються з часом стани компонентів, або вузлів через їх взаємодію з іншими вузлами, які з ними пов'язані. З'єднання представлені ланками мережі, де топологія мережі фіксується протягом усього часу. Стільникові автомати, булеві мережі та штучні нейронні мережі (без навчання) належать до цього класу.

МОДЕЛЮВАННЯ ДИНАМІКИ МЕРЕЖ ЗА ДОПОМОГОЮ ГРАФІВ

Оскільки NetworkX дозволяє задавати прості словники, як основну структуру даних, то ми можемо легко додавати стани до вузлів (та країв) та динамічно оновлювати ці стани ітераційно. Це і є моделюванням динаміки в мережах. Цей клас динамічних мережевих моделей описує динамічні зміни стану, що відбуваються на статичній топології мережі. Багато реальних динамічних мереж потрапляють до цієї категорії, включаючи:

- Регуляторні відносини між генами та білками всередині клітини, де вузли - це гени та/або білки, а стан вузлів - рівні їх експресії.
- Екологічна взаємодія між видами в екосистемі, де вузли є видами, а стани вузлів - їх популяціями.
- Захворювання інфекцією в соціальних мережах, де вузли - це особи, а стан вузла - їх епідеміологічні стани (наприклад, сприйнятливі, інфіковані, одужали, імунізовані тощо).
- Поширення інформації/культури в організаційні/соціальних мережах, де вузли - це особи або спільноти, а стани вузлів - їх інформаційні/культурні стани.

МОДЕЛЮВАННЯ ДИНАМІКИ МЕРЕЖ ЗА ДОПОМОГОЮ ГРАФІВ

Розв'яжемо задачу про моделювання прийняття рішень більшістю у соціальній мережі.

У графі соціальної мережі (на прикладі графу клубу карате Захарі) кожна вершина представляє **людину**, яка має один з двох можливих станів або думок:

- **0 — синя думка**,
- **1 — червона думка**.

На початку кожен учасник випадково обирає свій початковий стан (0 або 1).

Далі на кожному кроці часу **всі вершини оновлюють свої стани одночасно** за таким правилом:

- Порахувати загальну кількість однодумців у своїх сусідів і себе.
- Обчислити частку стану **1** серед себе та своїх сусідів.
- Якщо ця частка > 0.5 → вершина стає **червоною** (1).
- Якщо < 0.5 → стає **синьою** (0).
- Якщо рівно 0.5 → випадковим чином вибирається між 0 та 1.

МОДЕЛЬ ВИБОРЦЯ

Прикладом моделі з дискретними станами є модель виборця. Фактично - це перегляд моделі динамічної мережі вибору більшості, розробленої вище. Так само, як і в попередній моделі, кожен вузол («виборець») приймає один із скінченних дискретних станів (скажімо, чорно-білий, або червоний і синій - ви можете розглядати це як політичну думку), але правило оновлення інше. Замість того, щоб усі вузли оновлювали свої стани одночасно на основі вибору локальної більшості, модель виборця розглядає лише одну подію передачі думки за один раз між парою підключених вузлів, які випадково обрані з мережі. У цьому сенсі це асинхронна динамічна мережева модель.

МОДЕЛЮВАННЯ ДИНАМІКИ МЕРЕЖ ЗА ДОПОМОГОЮ ГРАФІВ

Існує три незначні варіації того, як вибираються ці вузли:

Оригінальна версія («перетягування»): Спочатку вузол «слухач» вибирається випадковим чином з мережі, а потім вузол «динамік» вибирається випадковим чином з сусідів слухача.

Зворотна версія («поштовх») : Спочатку з мережі випадковим чином вибирається вузол «динамік», а потім випадковим чином вибирається вузол «слухача» з сусідів динаміка.

Edge based (симетрична) версія: Спочатку ребро вибирається випадковим чином з мережі, а потім дві кінцеві точки (вузли) краю випадковим чином призначаються як «динамік» і «слухач».

У будь-якому випадку вузол «слухач» копіює стан вузла «динамік». Це повторюється кілька разів, поки вся мережа не досягне консенсусу з гомогенізованим станом.

Розглянемо на прикладі версію для моделі з перетягуванням.

ЕПІДЕМІЧНА МОДЕЛЬ В МЕРЕЖІ

Ще одним прикладом дискретних моделей динамічної мережі стану є епідемічна модель в мережі. Розглянемо модель сприйнятливості-зараженості-чутливості (SIS). У моделі SIS є всього два стани: ***Сприйнятливий*** і ***Інфікований***.

Сприйнятливий вузол може заразитися від зараженого сусідського вузла з ймовірністю зараження p_i (на зараженого сусіда), в той час як заражений вузол може відновитися назад до чутливого вузла (тобто не набутий імунітет) з ймовірністю відновлення p_r. Цей параметр розміщує все в двійкових станах (сприйнятливий чи інфікований), як і у попередніх прикладах, тому можна переробляти коди, розроблені перед тим.

Q & A